

Comparison of DPC Algorithm Variants

Xiuyuan Lu

1 Introduction

This document compares the original Density Peak Clustering (DPC) algorithm with three modified versions developed for improved robustness and automation. The comparison covers mathematical formulations, implementation differences, and performance characteristics.

2 Mathematical Foundations

2.1 Original DPC Algorithm

The original DPC relies on two core metrics:

- **Local Density (ρ):**

$$\rho_i = \sum_{j \neq i} e^{-\left(\frac{d_{ij}}{d_c}\right)^2} \quad (1)$$

or alternatively using a cutoff kernel:

$$\rho_i = |\{j \mid d_{ij} < d_c\}| \quad (2)$$

- **Minimum Distance to Higher Density (δ):**

$$\delta_i = \begin{cases} \min_{j: \rho_j > \rho_i} d_{ij} & \text{if } \exists j, \rho_j > \rho_i \\ \max_j d_{ij} & \text{otherwise} \end{cases} \quad (3)$$

Cluster centers are manually selected from the decision graph ($\gamma_i = \rho_i \times \delta_i$).

2.2 Modified Versions

Key improvements in your versions:

- **Density Calculation:**

$$\rho_i = \frac{1}{\text{percentile}/\text{mean}(\text{KNN distances}) + \epsilon} \quad (4)$$

- **Automated Center Selection:**

$$n_{\text{clusters}} = \text{KneeLocator}(\gamma \text{ curve}) \quad (5)$$

3 Detailed Comparison

3.1 Density Calculation

Table 1: Density Calculation Methods

Version	Formula	Parameters	Characteristics
Original DPC	$\rho_i = \sum e^{-(d_{ij}/d_c)^2}$	Manual d_c	Sensitive to cutoff
Initial	Same as original	Auto d_c (15% percentile)	Partial automation
Modified	1/KNN percentile(25%)	$k = 2$ neighbors	Local adaptation
Final	1/mean(KNN distances)	$k = 5$ neighbors	Smoother but outlier-sensitive

3.2 Cluster Center Selection

Table 2: Center Selection Strategies

Version	Method	Automation
Original	Manual selection from decision graph	None
Initial	Top 8 γ points	Fixed count
Modified	Top 2 + distance-constrained selection	Semi-automatic
Final	KneeLocator + target range (8-9)	Full automation

3.3 Noise Handling

Table 3: Noise Processing Approaches

Version	Method	Robustness
Original	No explicit handling	Noise affects results
Initial	Nearest non-noise assignment	Basic filtering
Modified/Final	Density threshold ($\rho > \bar{\rho}$) + nearest neighbor	Improved filtering

4 Performance Characteristics

Table 4: Overall Comparison

Feature	Original	Initial	Modified/Final
Parameter sensitivity	High (manual d_c)	Medium	Low
Automation level	None	Partial	High
Noise robustness	Poor	Medium	Good
High-dimension handling	Poor	Good (with PCA)	Excellent
Code complexity	Low	Medium	High/Medium

5 Conclusion

The evolutionary improvements demonstrate:

- Replacement of manual steps with data-driven automation
- Enhanced robustness through KNN-based density estimation
- Better noise handling via density thresholds
- Maintained interpretability while increasing applicability

The final version provides the best balance between automation and performance for practical applications.

6 Appendix

Here are the source codes for the original version, the modified version, and the final version of the DPC algorithm.

1: original version

```
import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
```

```

years = [2010, 2013, 2016, 2019, 2022]
indicators = [f'X{i}' for i in range(1, 9)]
column_names = ['State'] + indicators
states = (pd.read_excel('../raw_data/2022data.xlsx'))['State'].to_list()
plt.rcParams['font.family'] = 'Times New Roman'

# assign the noise points so that each country is in a certain cluster
def assign_noise_to_clusters(data, dpc_labels):
    # locate the noise
    noise_mask = (dpc_labels == -1)

    if not noise_mask.any(): # return the original label if the point is not a noise
        return dpc_labels

    # find the index and the label of the noise
    core_mask = ~noise_mask
    core_points = data[core_mask]
    core_labels = dpc_labels[core_mask]

    # locate the nearest neighbour
    nbrs = NearestNeighbors(n_neighbors=1).fit(core_points)
    distances, indices = nbrs.kneighbors(data[noise_mask])

    # assign the noise to a cluster
    dpc_labels[noise_mask] = core_labels[indices.flatten()]

    return dpc_labels

# perform the DPC process
def cluster_by_DPC(year):
    plt.clf()
    data = pd.read_excel(f'../processed_data/{year}_IDOCRIW_weighted_data.xlsx')[indicators]
    # Preparation for visualization
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(data)

    # Calculate the density and distances
    distances = squareform(pdist(data))

    # Set cutoff distance (dc) with a minimum value to avoid division by zero
    dc = max(np.percentile(distances, 15), 1e-5) # Ensure dc is not too small

    # Calculate density with a small epsilon to avoid division by zero
    epsilon = 1e-10
    density = np.sum(np.exp(-((distances + epsilon) / dc) ** 2), axis=1)

    # Calculate delta (minimum distance to a point with higher density)
    delta = np.zeros_like(density)
    for i in range(len(data)):
        higher_density_mask = density > density[i]
        if np.any(higher_density_mask):
            delta[i] = np.min(distances[i, higher_density_mask])
        else:
            delta[i] = np.max(distances[i]) # Assign the maximum distance if no higher density
            # → points

    # Select the centroids of clusters
    n_clusters = 8 # Assume 5 clusters
    centers = np.argsort(-density * delta)[:n_clusters]

    # Clustering based on k-means
    kmeans = KMeans(n_clusters=n_clusters, init=data.iloc[centers])
    dpc_labels = kmeans.fit_predict(data)

    # Assign the noise manually
    dpc_labels = assign_noise_to_clusters(data, dpc_labels)

    # Visualization of the clustering
    plt.scatter((X_pca[:, 0], X_pca[:, 1]), c=dpc_labels, cmap='viridis', marker='o', edgecolor='k')
    plt.title(f'{year} DPC Clustering')
    plt.colorbar(label='Cluster Label')
    plt.tight_layout()

    # Annotate each point with its sample name
    sample_names = states
    for i, name in enumerate(sample_names):

```

```

plt.text(X_pca[i, 0], X_pca[i, 1], name, fontsize=8, ha='left')

plt.savefig(f'../visualization/DPC/{year}_DPC.jpg', bbox_inches='tight', dpi=400)

# Group results by cluster
score = pd.read_excel(f'../processed_data/{year}_DOBI.xlsx')
unique_labels = set(dpc_labels)
clusters = {label: [] for label in unique_labels}

for i, name in enumerate(sample_names):
    cluster_id = dpc_labels[i]
    clusters[cluster_id].append(name)

cluster_df = {i: [clusters[i]] for i in unique_labels}
cluster_df = pd.DataFrame(cluster_df).transpose()
cluster_df.columns = ['State']
cluster_df['feature'] = cluster_df['State'].apply(lambda x: np.mean(score[score['State'].isin(x)
    ↪ ]['Ranking'])))
cluster_df = cluster_df.sort_values(by='feature', ascending=True, ignore_index=True)
cluster_df['real_label'] = list(range(1, len(unique_labels) + 1))

sorted_clusters = dict(zip(cluster_df['real_label'].to_list(), cluster_df['State'].to_list()))

# Save all clusters into a single file
dpc_clusters_df = pd.concat([pd.DataFrame({'State': samples, 'Group': [cluster_id] * len(samples)
    ↪ }) for cluster_id, samples in sorted_clusters.items()])
print(dpc_clusters_df)
dpc_clusters_df = pd.merge((pd.read_excel('../raw_data/2022data.xlsx'))['State'],
    ↪ dpc_clusters_df, how='left', on='State')
dpc_clusters_df.to_excel(f'../processed_data/{year}_DPC_groups.xlsx', index=False)

for year in years:
    cluster_by_DPC(year)

```

2: modified version

```

import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score
from kneed import KneeLocator

# Configuration and initialization
years = [2010, 2013, 2016, 2019, 2022]
indicators = [f'X{i}' for i in range(1, 9)]
states = (pd.read_excel('../raw_data/2022data.xlsx'))['State'].to_list()
plt.rcParams['font.family'] = 'Times New Roman'

def assign_noise_to_clusters(data, labels, density=None, max_noise_ratio=0.2):
    """Assign noise points to nearest non-noise clusters, considering density."""
    noise_mask = (labels == -1)
    if not noise_mask.any() or np.sum(noise_mask) / len(data) > max_noise_ratio:
        return labels

    if density is not None:
        noise_mask = noise_mask & (density > np.mean(density))

    core_points = data[~noise_mask]
    core_labels = labels[~noise_mask]
    nbrs = NearestNeighbors(n_neighbors=1).fit(core_points)
    _, indices = nbrs.kneighbors(data[noise_mask])
    labels[noise_mask] = core_labels[indices.flatten()]
    return labels

def merge_small_clusters(data, labels, min_size=1):
    """Merge clusters smaller than min_size into nearest larger clusters."""
    unique_labels, counts = np.unique(labels, return_counts=True)
    small_clusters = unique_labels[counts < min_size]

    for sc in small_clusters:

```

```

        mask = (labels == sc)
        if np.sum(mask) == 0:
            continue
        nbrs = NearestNeighbors(n_neighbors=1).fit(data[~mask])
        _, indices = nbrs.kneighbors(data[mask])
        nearest_labels = labels[~mask][indices.flatten()]
        labels[mask] = nearest_labels
    return labels

def enforce_cluster_count(gamma, target_range=(5, 6)):
    """
    Modified version to better hit target range by:
    1. Using a more flexible knee detection
    2. Adding a fallback to select top candidates when knee detection fails
    """
    gamma_sorted = -np.sort(-gamma)

    # Try knee detection first
    kneedle = KneeLocator(range(len(gamma_sorted)), gamma_sorted,
                          curve='convex', direction='decreasing')

    # If we get a clear knee point
    if kneedle.elbow is not None:
        n_clusters = kneedle.elbow + 1
        if n_clusters < target_range[0]:
            # If too few, take top candidates from target range
            return min(target_range[1], len(gamma_sorted))
        elif n_clusters > target_range[1]:
            # If too many, take midpoint
            return target_range[1]
        else:
            return n_clusters
    else:
        # Fallback: Take midpoint of target range
        return min(target_range[1], len(gamma_sorted) // 2 + 1)

def cluster_by_DPC(year):
    """Modified clustering function with enhanced center selection"""
    plt.clf()

    # 1. Load and prepare data
    data = pd.read_excel(f'../processed_data/{year}_IDOCRIW_weighted_data.xlsx')[indicators]
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(data)

    # 2. Improved density calculation
    k = 2 # Number of neighbors for density estimation
    nbrs = NearestNeighbors(n_neighbors=k).fit(data)
    knn_distances, _ = nbrs.kneighbors(data)
    # Adjust the density calculation to be more discriminative
    density = 1 / (np.percentile(knn_distances, 25, axis=1) + 1e-10) # Use 25th percentile distance

    # 3. Calculate delta
    distances = squareform(pdist(data))
    delta = np.zeros_like(density)
    for i in range(len(data)):
        higher_density = density > density[i]
        if np.any(higher_density):
            delta[i] = np.min(distances[i, higher_density])
        else:
            delta[i] = np.max(distances[i])

    # 4. Enforce 8-9 clusters
    gamma = density * delta
    n_clusters = enforce_cluster_count(gamma, target_range=(8, 9))

    # Enhanced center selection
    centers = []
    gamma_sorted_idx = np.argsort(-gamma)

    # First select the top 2 most obvious centers
    centers.extend(gamma_sorted_idx[:2])

    # Then select remaining centers spaced apart

```

```

min_distance = np.median(distances) * 0.5 # Minimum distance between centers
for i in range(2, len(gamma_sorted_idx)):
    if len(centers) >= n_clusters:
        break
    candidate = gamma_sorted_idx[i]
    # Check distance to existing centers
    if all(distances[candidate, c] > min_distance for c in centers):
        centers.append(candidate)

# If we didn't get enough centers, add the next best candidates
while len(centers) < n_clusters and len(centers) < len(gamma_sorted_idx):
    next_candidate = gamma_sorted_idx[len(centers)]
    centers.append(next_candidate)

# Modified K-Means initialization
if len(centers) >= n_clusters:
    kmeans = KMeans(n_clusters=n_clusters, init=data.iloc[centers[:n_clusters]],
                    n_init=1, random_state=42)
else:
    kmeans = KMeans(n_clusters=n_clusters, init='k-means++',
                    n_init=10, random_state=42)

dpc_labels = kmeans.fit_predict(data)

# 7. Post-processing
dpc_labels = merge_small_clusters(data, dpc_labels, min_size=1) # Allow smaller min_size since
    ↪ we enforce count
dpc_labels = assign_noise_to_clusters(data, dpc_labels, density)

# 8. Evaluation
if len(np.unique(dpc_labels)) > 1:
    silhouette_avg = silhouette_score(data, dpc_labels)
    print(f"{year} Silhouette Score: {silhouette_avg:.3f}")

# 9. Visualization
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dpc_labels, cmap='viridis', marker='o', edgecolor='k')
plt.title(f'{year} DPC Clustering')
plt.colorbar(label='Cluster Label')
for i, name in enumerate(states):
    plt.text(X_pca[i, 0], X_pca[i, 1], name, fontsize=8, ha='left')
plt.savefig(f'../visualization/DPC/{year}_DPC_modified.jpg', bbox_inches='tight', dpi=400)
plt.close()

# 10. Save results
score = pd.read_excel(f'../processed_data/{year}_DOBI.xlsx')
unique_labels = set(dpc_labels)
clusters = {label: [] for label in unique_labels}

for i, name in enumerate(states):
    clusters[dpc_labels[i]].append(name)

# Sort clusters by average ranking
cluster_df = pd.DataFrame({
    'State': [clusters[label] for label in unique_labels],
    'Label': list(unique_labels)
})
cluster_df['AvgRank'] = cluster_df['State'].apply(
    lambda x: np.mean(score[score['State'].isin(x)]['Ranking']))
cluster_df = cluster_df.sort_values('AvgRank').reset_index(drop=True)
cluster_df['Group'] = range(1, len(cluster_df) + 1)

# Expand to one row per state
result_df = pd.concat([
    pd.DataFrame({'State': states, 'Group': group})
    for states, group in zip(cluster_df['State'], cluster_df['Group'])
])
print(result_df)
result_df = pd.merge((pd.read_excel('../raw_data/2022data.xlsx'))['State'], result_df, how='left',
    ↪ on='State')
result_df.to_excel(f'../processed_data/{year}_DPC_modified_groups.xlsx', index=False)

# Execute clustering for all years
for year in years:
    cluster_by_DPC(year)

```

```

import numpy as np
import pandas as pd
from scipy.spatial.distance import pdist, squareform
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score
from kneed import KneeLocator

# Configuration and initialization
years = [2010, 2013, 2016, 2019, 2022]
indicators = [f'X{i}' for i in range(1, 9)]
states = (pd.read_excel('../raw_data/2022data.xlsx'))['State'].to_list()
plt.rcParams['font.family'] = 'Times New Roman'

def assign_noise_to_clusters(data, labels, density=None, max_noise_ratio=0.2):
    """Assign noise points to nearest non-noise clusters, considering density."""
    noise_mask = (labels == -1)
    if not noise_mask.any() or np.sum(noise_mask) / len(data) > max_noise_ratio:
        return labels

    if density is not None:
        noise_mask = noise_mask & (density > np.mean(density))

    core_points = data[~noise_mask]
    core_labels = labels[~noise_mask]
    nbrs = NearestNeighbors(n_neighbors=1).fit(core_points)
    _, indices = nbrs.kneighbors(data[noise_mask])
    labels[noise_mask] = core_labels[indices.flatten()]
    return labels

def enforce_cluster_count(gamma, target_range=(8, 9)):
    """Ensure we get clusters within target range."""
    gamma_sorted = -np.sort(-gamma)
    kneedle = KneeLocator(range(len(gamma_sorted)), gamma_sorted,
                          curve='convex', direction='decreasing')

    if kneedle.elbow is not None:
        n_clusters = kneedle.elbow + 1
        if n_clusters < target_range[0]:
            return target_range[0]
        elif n_clusters > target_range[1]:
            return target_range[1]
        return n_clusters
    return target_range[0] # Default to minimum target

def cluster_by_DPC(year):
    """Main DPC clustering function with removed small cluster merging."""
    plt.clf()
    data = pd.read_excel(f'../processed_data/{year}_IDOCRIW_weighted_data.xlsx')[indicators]
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(data)

    # 1. Density calculation
    k = 5
    nbrs = NearestNeighbors(n_neighbors=k).fit(data)
    knn_distances, _ = nbrs.kneighbors(data)
    density = 1 / (np.mean(knn_distances, axis=1) + 1e-10)

    # 2. Delta calculation
    distances = squareform(pdist(data))
    delta = np.zeros_like(density)
    for i in range(len(data)):
        higher_density = density > density[i]
        if np.any(higher_density):
            delta[i] = np.min(distances[i, higher_density])
        else:
            delta[i] = np.max(distances[i])

    # 3. Cluster count selection
    gamma = density * delta
    n_clusters = enforce_cluster_count(gamma)

```

```

# 4. Center selection
centers = np.argsort(-gamma)[:n_clusters]
kmeans = KMeans(n_clusters=n_clusters, init=data.iloc[centers],
                n_init=1, random_state=42)
dpc_labels = kmeans.fit_predict(data)

# 5. Only assign noise (no cluster merging)
dpc_labels = assign_noise_to_clusters(data, dpc_labels, density)

# Evaluation
if len(np.unique(dpc_labels)) > 1:
    silhouette_avg = silhouette_score(data, dpc_labels)
    print(f"{year} Silhouette Score: {silhouette_avg:.3f} (n_clusters={n_clusters})")
    print(f"Actual unique clusters: {len(np.unique(dpc_labels))}")

# Visualization
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=dpc_labels, cmap='viridis', marker='o', edgecolor='k')
plt.title(f"{year} DPC Clustering")
plt.colorbar(label='Cluster Label')
for i, name in enumerate(states):
    plt.text(X_pca[i, 0], X_pca[i, 1], name, fontsize=8, ha='left')
plt.savefig(f'../visualization/DPC/{year}_DPC_remodified.jpg', bbox_inches='tight', dpi=400)
plt.close()

# 10. Save results
score = pd.read_excel(f'../processed_data/{year}_DOBI.xlsx')
unique_labels = set(dpc_labels)
clusters = {label: [] for label in unique_labels}

for i, name in enumerate(states):
    clusters[dpc_labels[i]].append(name)

# Sort clusters by average ranking
cluster_df = pd.DataFrame({
    'State': [clusters[label] for label in unique_labels],
    'Label': list(unique_labels)
})
cluster_df['AvgRank'] = cluster_df['State'].apply(
    lambda x: np.mean(score[score['State'].isin(x)]['Ranking']))
cluster_df = cluster_df.sort_values('AvgRank').reset_index(drop=True)
cluster_df['Group'] = range(1, len(cluster_df) + 1)

# Expand to one row per state
result_df = pd.concat([
    pd.DataFrame({'State': states, 'Group': group})
    for states, group in zip(cluster_df['State'], cluster_df['Group'])
])
print(result_df)
result_df = pd.merge((pd.read_excel('../raw_data/2022data.xlsx'))['State'], result_df, how='left',
                    on='State')
result_df.to_excel(f'../processed_data/{year}_DPC_remodified_groups.xlsx', index=False)

# Execute clustering
for year in years:
    cluster_by_DPC(year)

```
